CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 1

Name: Kanishk Jamwal
UID: 24BAI70827
Branch: CSE AIML (CS221)
Section/Group: 24AIT_NTPP-3[A]
Semester: 5th
Date of Performance: 08/07/2026
Subject: Full Stack-II
Subject Code: 24CSP-337

EXPERIMENT – 1.1

1. **AIM:** To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

2. OBJECTIVE:

1. To understand multi-platform content handling.
2. To implement real-time character validation.
3. To design a responsive and user-friendly interface.

3. SOFTWARE REQUIREMENTS:

1. React.js
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code

4. THEORY

React.js is an open-source JavaScript library developed by Meta (Facebook) for building modern, fast, and interactive user interfaces. It follows a component-based architecture, where the application is divided into reusable components, making the code easier to develop, maintain, and scale. React also uses a Virtual DOM, which improves application performance by updating only the changed parts of the webpage instead of reloading the entire page.

In this experiment, a Social Media Post Composer is developed that supports multiple platforms such as Twitter, LinkedIn, and Instagram, each having its own character limit. The application uses the **useState()** Hook to manage the selected platform and the content entered by the user. As the user types in the **textarea**, the application continuously counts the number of characters and compares it with the predefined limit of the selected platform. Using conditional rendering, the application displays either "Ready to Post" or "Character Limit Exceeded" based on the validation result. User interactions

are handled through the **onChange** event, making the interface dynamic and responsive. This experiment provides practical knowledge of React components, JSX, state management, controlled components, event handling, and real-time validation, which are essential concepts for developing modern frontend web applications.

5. IMPLEMENTATION/PROCEDURE

1. Install Node.js and verify the installation.
2. Create a React project using `npx create-react-app`.
3. Open the project in Visual Studio Code.
4. Run the application using `npm start`.
5. Design the Post Composer interface.
6. Add platform selection (Twitter, LinkedIn, Instagram).
7. Implement `useState()` for managing platform and post.
8. Display live character count.
9. Validate character limits for each platform.
10. Display appropriate validation messages and test the application.

6. CODE:

```
import { useState } from "react";

function App() {
  const [platform, setPlatform] = useState("Twitter");
  const [post, setPost] = useState("");

  const limits = {
    Twitter: 280,
    LinkedIn: 3000,
    Instagram: 2200,
  };

  const limit = limits[platform];
  const isExceeded = post.length > limit;

  return (
    <div>
      <h2>Social Media Post Composer</h2>

      <select
        value={platform}
        onChange={(e) => setPlatform(e.target.value)}
      >
        <option>Twitter</option>
        <option>LinkedIn</option>
```

```
        <option>Instagram</option>
      </select>

      <br /><br />

      <textarea
        rows="6"
        placeholder="Write your post..."
        value={post}
        onChange={(e) => setPost(e.target.value)}
      ></textarea>

      <p>Characters: {post.length} / {limit}</p>

      {isExceeded ? (
        <p style={{ color: "red" }}>Character Limit Exceeded!</p>
      ) : (
        <p style={{ color: "green" }}>Ready to Post</p>
      )}

      <button>Save Draft</button>
    </div>
  );
}
export default App;
```

7. OUTPUT

The application successfully displays a Post Composer interface where users can select a social media platform, write a post, view the live character count, and receive instant validation based on the selected platform's character limit.

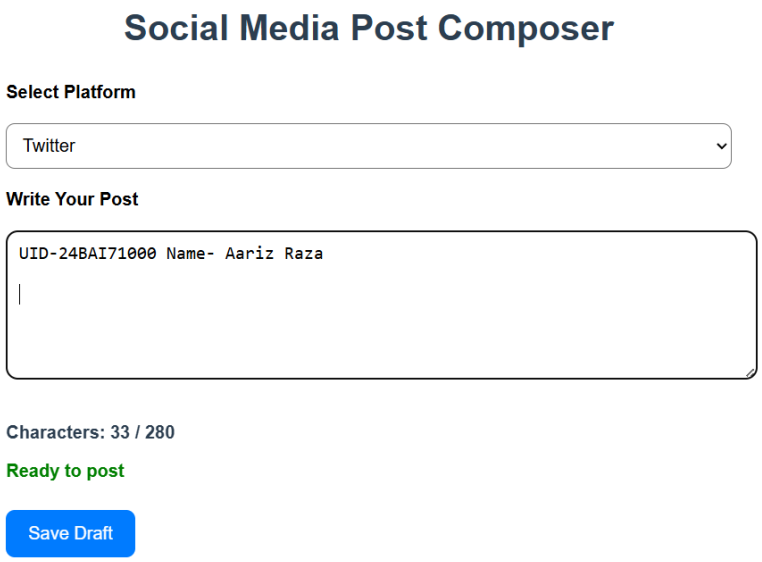


Fig 1.1: Social Media Post Composer

8. LEARNING OUTCOME

- Understand the fundamentals of React.js.
- Create and run a React application.
- Design a dynamic user interface.
- Implement state management using `useState()`.
- Handle user input using controlled components.
- Perform real-time character validation.
- Apply conditional rendering in React.
- Develop responsive frontend applications.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 1.2

1. **AIM:** To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.

2. OBJECTIVE:

1. To manage draft data using frontend state.
2. To implement CRUD operations for draft handling.
3. To understand asynchronous workflows in UI applications.

3. SOFTWARE REQUIREMENTS:

1. Node.js (LTS Version)
2. npm (Node Package Manager)
3. Visual Studio Code
4. React.js

4. THEORY

A Draft Management System is a feature that allows users to temporarily save their work before publishing it. It improves the user experience by preventing data loss and enabling users to edit, update, or delete their drafts whenever required. In modern web applications, draft management is commonly used in email services, blogging platforms, and social media applications.

In this experiment, a Draft Management System is developed using React.js. The application uses the **`useState()`** Hook to store and manage multiple drafts dynamically. Users can write a post and save it as a draft. The saved drafts are displayed using the **`map()`** function, while the **`filter()`** function is used to delete unwanted drafts. The Edit feature allows users to modify an existing draft by loading it back into the input field. To retain the drafts even after refreshing the browser, Local Storage is used along with the **`useEffect()`** Hook, which loads the saved data when the application starts. This experiment demonstrates important React concepts such as state management, CRUD operations, event handling, array manipulation, dynamic rendering, and browser storage, which are essential for developing modern frontend applications.

5. IMPLEMENTATION/PROCEDURE

1. Continue from Experiment 1.1.1.
2. Create a state variable to store multiple drafts.
3. Implement the Save Draft functionality using `useState()`.
4. Display all saved drafts using the `map()` function.
5. Implement Edit Draft functionality.
6. Implement Delete Draft functionality using `filter()`.
7. Store drafts in Local Storage.
8. Load saved drafts automatically using `useEffect()`.
9. Test all CRUD operations.
10. Verify that drafts remain after refreshing the browser.

6. CODE

```
import { useState } from "react";

function App() {
  const [post, setPost] = useState("");
  const [drafts, setDrafts] = useState([]);

  const saveDraft = () => {
    if (post.trim() === "") return;
    setDrafts([...drafts, post]);
    setPost("");
  };

  const editDraft = (index) => {
    setPost(drafts[index]);
    setDrafts(drafts.filter((_, i) => i !== index));
  };

  const deleteDraft = (index) => {
    setDrafts(drafts.filter((_, i) => i !== index));
  };

  return (
    <div>
      <h2>Draft Management System</h2>

      <textarea
        value={post}
        onChange={(e) => setPost(e.target.value)}
        placeholder="Write your post..."
      ></textarea>

      <br /><br />

      <button onClick={saveDraft}>Save Draft</button>

      <h3>Saved Drafts</h3>

      {drafts.map((draft, index) => (
        <div key={index}>
          <p>{draft}</p>
```

```
        <button onClick={() => editDraft(index)}>Edit</button>
        <button onClick={() => deleteDraft(index)}>Delete</button>
      </div>
    ))}
  </div>
 );
}

export default App;
```

7. OUTPUT (SCREENSHOT OF WEB + explain)

The application successfully allows users to write posts, save them as drafts, display all saved drafts dynamically, edit existing drafts, and delete unwanted drafts. The draft list updates automatically whenever changes are made, providing an interactive and user-friendly experience.

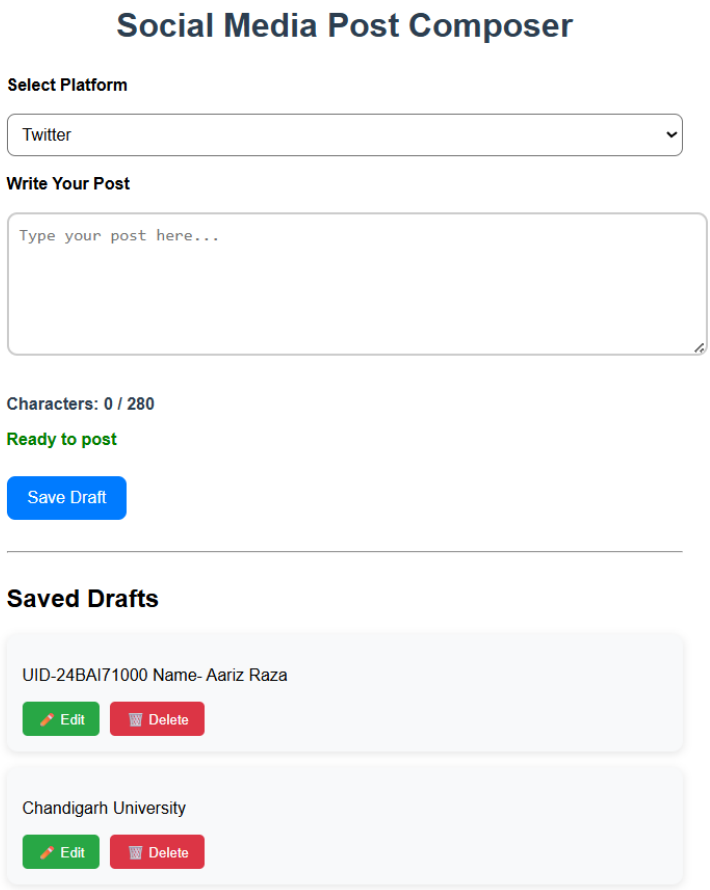


Fig 1.2: Draft Management System Webpage

8. LEARNING OUTCOME

- Understand the concept of Draft Management.
- Implement CRUD operations in React.
- Manage application data using `useState()`.
- Display dynamic data using the `map()` function.
- Modify arrays using the `filter()` function.
- Use the `useEffect()` Hook for lifecycle management.
- Store and retrieve data using Local Storage.
- Develop interactive and responsive frontend applications using React.